**Universidad Autónoma de Chiriquí**
**Facultad de Economía**
**Escuela De Ciencias de Computación**

**Lic. en Gestión de Tecnología de la Información**

**“Programación V – PROG 309”**

**Estudiante:**
**Arian Cervantes**
**Jairo Chávez**
**Débora Cisneros**
**Daniela Pittí**

**Prof. Alexis Flores**

# Laboratorio #5

***Caso: Sistema de gestión de inventario — Definición del escenario y los requerimientos***

**II Semestre**

# 1. Objetivo del laboratorio

Analizar el caso de estudio de un sistema de gestión de inventario para una empresa de distribución y redactar, en equipo, la descripción del escenario —actores, objetivos y alcance— que servirá como base del proyecto que se desarrollará durante todo el Eje Temático N.° 2 con el patrón de capas, proponiendo además la distribución preliminar de responsabilidades entre las capas de presentación, negocio y datos.

# 2. Resumen del caso de estudio

Una empresa de distribución maneja actualmente su inventario en hojas de cálculo compartidas. Con el crecimiento del negocio han comenzado a ocurrir errores de conteo, productos agotados sin previo aviso y descuadres entre el inventario físico y el registrado. La empresa necesita un sistema que registre productos, controle entradas y salidas en tiempo real y genere reportes de existencias confiables para la toma de decisiones.

El caso plantea tres necesidades determinantes para la definición del escenario:

- Eliminar los errores de conteo y los descuadres entre el inventario físico y el registrado.
- Evitar que los productos se agoten sin previo aviso.
- Contar con reportes de existencias confiables que respalden la toma de decisiones.

# 3. Descripción del escenario

## 3.1 Descripción general

La empresa distribuye productos a sus clientes y hoy controla el inventario en hojas de cálculo compartidas, donde varias personas editan los mismos archivos al mismo tiempo, sin validaciones y sin historial confiable de los cambios. A medida que el negocio crece, esto provoca errores de conteo, existencias registradas que no coinciden con las físicas y productos que se agotan sin que nadie lo advierta a tiempo, con el consecuente retraso en despachos y pérdida de ventas.

El sistema propuesto centralizará el inventario en una aplicación con base de datos única, donde cada producto estará registrado una sola vez, cada entrada o salida de mercancía actualizará las existencias de inmediato y el sistema impedirá operaciones inconsistentes (por ejemplo, despachar más unidades de las disponibles). Además, generará alertas de stock mínimo y reportes de existencias confiables para apoyar las decisiones de reabastecimiento y de gestión de la empresa.

## 3.2 Actores del sistema

| Actor | Responsabilidad |
|---|---|
| Administrador del sistema | Crea y gestiona los usuarios, asigna roles y permisos, y configura los parámetros generales (categorías de productos y niveles de stock mínimo). |
| Encargado de almacén | Registra y actualiza los productos, registra las entradas de mercancía recibida y realiza la verificación del conteo físico frente a las existencias del sistema. |
| Personal de despacho | Registra las salidas de mercancía para los pedidos de los clientes y consulta la disponibilidad de productos antes de despachar. |
| Gerente / responsable de compras | Consulta los reportes de existencias y las alertas de stock mínimo para decidir qué productos reabastecer y en qué momento. |

## 3.3 Objetivos del sistema

- Mantener un registro único y completo de los productos, de modo que el 100 % de los artículos del inventario tenga código, nombre, categoría, existencia y stock mínimo definidos.
- Actualizar las existencias en tiempo real, reflejando cada entrada y salida en el momento en que se registra.
- Reducir los descuadres entre el inventario físico y el registrado, impidiendo salidas que superen la existencia disponible y conservando el historial de cada movimiento con su fecha y responsable.
- Evitar los agotados sin previo aviso, generando una alerta automática cuando un producto alcance su nivel de stock mínimo.
- Ofrecer reportes de existencias confiables y actualizados, consultables en cualquier momento, que sirvan de base para la toma de decisiones.
- Sustituir las hojas de cálculo compartidas como fuente oficial del inventario de la empresa.

### 3.4 Alcance

| Dentro del alcance | Fuera del alcance |
|---|---|
| Registro, consulta y edición de productos (código, nombre, categoría, existencia, stock mínimo). | Facturación, cobros y cuentas por cobrar. |
| Registro de entradas y salidas de mercancía con actualización inmediata de existencias. | Gestión de órdenes de compra y de proveedores. |
| Historial de movimientos por producto (fecha, tipo, cantidad y usuario). | Contabilidad y valoración financiera del inventario. |
| Alertas de stock mínimo. | Logística de transporte y rutas de distribución. |
| Reportes de existencias y de movimientos. | Integración con tiendas en línea, ERP o sistemas externos. |
| Control de acceso por usuario y rol. | Aplicación móvil y lectura de códigos de barras (posibles versiones futuras). |

### 3.5 Capas identificadas

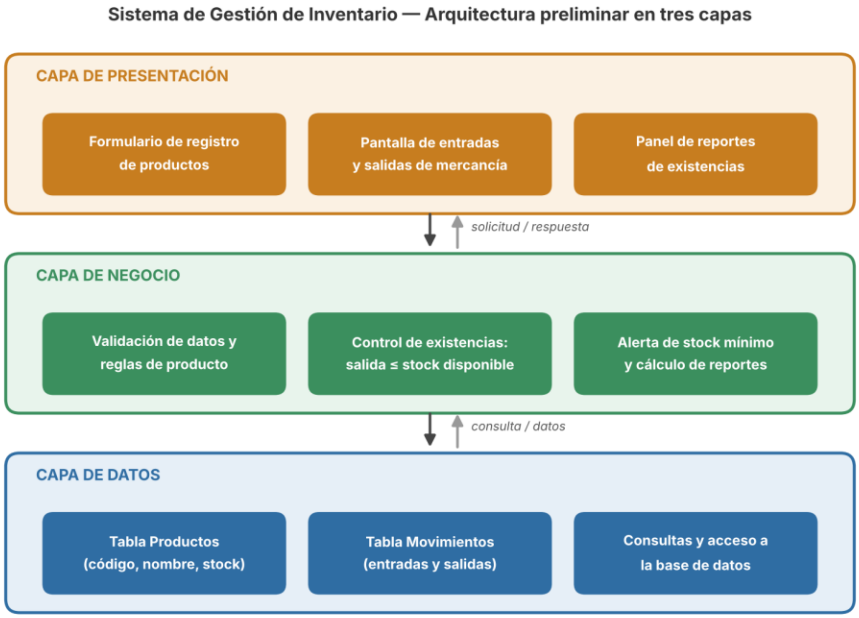
| Capa | Qué incluirá en este caso |
|---|---|
| Presentación | Interfaz gráfica desarrollada en Java con la que interactúan los usuarios: formulario de registro de productos, pantalla de entradas y salidas de mercancía, panel de alertas y pantalla de reportes de existencias. Solo muestra información y recoge datos; no contiene reglas de negocio ni consultas a la base de datos. |
| Negocio | Clases Java con las reglas del sistema: validación de los datos del producto, verificación de que una salida no supere la existencia disponible, actualización de existencias en cada movimiento, detección de stock mínimo, cálculo de los reportes y control de permisos según el rol del usuario. |
| Datos | Base de datos MySQL (administrada con XAMPP) con las tablas de productos, movimientos de inventario y usuarios, junto con las clases Java de acceso a datos que realizan las operaciones de inserción, consulta y actualización que utiliza la capa de negocio. |

Para el desarrollo se utilizará el lenguaje Java con el entorno de desarrollo Eclipse, y XAMPP para disponer del servidor Apache y la base de datos MySQL en la que se almacenará la información del inventario.

## 4. Diagrama preliminar de las tres capas

El siguiente diagrama muestra las tres capas identificadas con un ejemplo concreto de componente por cada una (los tres componentes de cada capa corresponden a las funciones descritas en la

sección 3.5). Cada capa solo se comunica con la inmediatamente adyacente: la presentación con el negocio y el negocio con los datos, pero nunca la presentación directamente con los datos.




## 5. Desafío cognitivo

¿Qué problemas podría causar mezclar la lógica de negocio con el acceso a datos en una sola capa?

Si en el sistema de inventario la regla “no se puede despachar más de lo que hay en existencia” estuviera escrita dentro de las mismas consultas a la base de datos, cada pantalla (registro de salidas, reportes, alertas) tendría que repetirla, y un cambio en la regla obligaría a modificar código en muchos lugares, con el riesgo de olvidar alguno y volver a generar descuadres, justo el problema que la empresa quiere resolver. Además, no se podría probar la regla de forma aislada sin tener conectada la base de datos real, lo que dificulta detectar errores a tiempo. También se pondría en riesgo la seguridad, porque la validación y los permisos quedarían mezclados con el acceso directo a los datos y sería más fácil saltarse controles. Por último, si la empresa decidiera cambiar de gestor de base de datos o crecer en volumen, habría que reescribir también las reglas de negocio, de modo que el mantenimiento y la escalabilidad del sistema serían mucho más costosos.

## 6. Conclusiones

- Definir primero el escenario permite que todo el equipo comparta una misma comprensión del problema de negocio —en este caso los descuadres, los agotados sin aviso y la falta de reportes confiables— antes de escribir código, evitando construir una solución para un problema distinto.

- Identificar los actores y la responsabilidad concreta de cada uno aclara quién usará el sistema y para qué, lo que sirve de base para especificar después los requisitos funcionales y los permisos de cada rol.
- Establecer explícitamente qué entra y qué queda fuera del alcance de la primera versión protege al proyecto de crecer sin control y deja claro qué se entregará, qué es verificable y qué se dejará para versiones futuras.
- Tener claro el escenario antes de diseñar la arquitectura hace posible decidir con criterio qué corresponde a la capa de presentación, a la de negocio y a la de datos, lo que da como resultado un sistema más fácil de mantener, probar y hacer crecer.

## 7. Referencias

Huaman, W. C. (2018). *Los 10 patrones comunes de arquitectura de software*. Medium.

Blancarte, O. (2020). *Introducción a la arquitectura de software*.